



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II 2025/2026

(SECP3133) HIGH PERFORMANCE DATA PROCESSING

SECTION 02

PROJECT 2:

**REAL-TIME SENTIMENT ANALYSIS OF MAE MOBILE BANKING REVIEWS
USING APACHE KAFKA AND SPARK**

LECTURER: DR. SEAH CHOON SEN

GROUP: DATADUCKS

STUDENT NAME	MATRIC NO
GUI KAH SIN	A23CS0080
SABRINA HENG WEI QI	A23CS0265
LING YU QIAN	A23CS0301
WOO CHENG SHUAN	A23CS0283

Table of Contents

1.0 Introduction.....	4
1.1 Background.....	4
1.2 Objectives.....	4
1.3 Scope	5
2.0 Data Acquisition & Preprocessing.....	6
2.1 Data Source.....	6
2.2 Development Tools and Libraries	6
2.3 Data Collection Process	7
2.4 Data Validation	8
2.5 Text Preprocessing	10
2.5.1 Lowercase Conversion	10
2.5.2 URL Removal.....	11
2.5.3 HTML Tag Removal.....	11
2.5.4 Emoji Removal	11
2.5.5 Punctuation Removal	12
2.5.6 Extra Space Removal	12
2.5.7 Tokenization	13
2.5.8 Stopword Removal	13
2.5.9 Lemmatization	14
3.0 Sentiment Model Development	16
3.1 Dataset Preparation	16
3.2 Data Splitting	17
3.3 Model 1: Naïve Bayes with TF-IDF	17
3.3.1 Feature Extraction	17
3.3.2 Model Training	18
3.3.3 Evaluation.....	18
3.4 Model 2: Long Short-Term Memory (LSTM) Network	20
3.4.1 Text Tokenization and Sequence Encoding	20
3.4.2 Handling Class Imbalance	20
3.4.3 Model Architecture	21
3.4.4 Model Training	21

3.4.5 Evaluation.....	22
3.5 Model Comparison.....	24
4.0 Apache System Architecture.....	25
5.0 Analysis & Results	26
6.0 Optimization & Comparison	28
6.1 Naive Bayes vs LSTM.....	28
6.2 Compare model and pipeline performance between batch mode and streaming mode	29
7.0 Dashboard Visualization and Analysis	30
8.0 Conclusion & Future Work	33
9.0 Reference.....	34

1.0 Introduction

1.1 Background

The rapid growth of digital banking services has transformed the way customers interact with financial institutions. Mobile banking applications have become an essential platform for performing daily financial activities such as fund transfers, bill payments, QR code transactions, investment management and account monitoring. As the number of users continues to increase, customers frequently express their experiences, opinions and satisfaction levels by submitting reviews through application marketplace such as Google Play Store. These reviews contain valuable information that reflects users' perceptions of application performance, usability, reliability, security and service quality.

However, the large volume of user-generated reviews makes manual analysis inefficient and time-consuming. Organizations require automated solutions that can continuously process and analyse customer feedback to identify user sentiment in real time. Sentiment analysis, a Natural Language Processing (NLP) technique, enables the automatic classification of textual data into positive, negative or neutral sentiments, allowing organizations to better understand customer opinions and respond more effectively to user concerns.

In modern big data environments, sentiment analysis is often integrated with distributed data processing technologies to support real-time analytics, Apache Kafka provides a scalable messaging platform for streaming incoming data, while Apache Spark enables high-performance distributed processing of large datasets. By combining these technologies with machine learning models, organizations can build an end-to-end real-time sentiment analysis pipeline capable of processing continuous streams of customer reviews efficiently.

This project focuses on analyzing customer reviews of the MAE by Maybank mobile banking application collected from the Google Play Store. The collected reviews are preprocessed using NLP techniques before being classified by sentiment analysis models. The processed results are then integrated into a real-time streaming architecture using Apache Kafka and Apache Spark, allowing sentiment trends to be monitored through interactive dashboards. This project demonstrates the practical application of data engineering, machine learning and big data technologies in solving real-world business problems.

1.2 Objectives

The objectives of this project are:

- To collect Malaysian-relevant customer reviews of the MAE by Maybank mobile application from the Google Play Store.
- To preprocess the collected textual data using Natural Language Processing (NLP) techniques, including text cleaning, tokenization, stopword removal and lemmatization.
- To develop and evaluate multiple sentiment classification models for identifying positive, negative and neutral customer sentiments.

- To implement a real-time data streaming pipeline using Apache Kafka and Apache Spark for processing incoming customer reviews.
- To store the processed sentiment data and visualize sentiment trends using interactive dashboards for analytical insights.

1.3 Scope

This project focuses on developing a real-time analysis system for customer review of the MAE by Maybank mobile banking application. The textual data is collected from publicly available Google Play Store reviews using the google-play-scraper library. The collected dataset is preprocessed through several NLP techniques, including lowercase conversion, URL removal, HTML tag removal, emoji removal, punctuation removal, tokenization, stopword removal and lemmatization to improve data quality before model training.

The cleaned dataset is then used to train and evaluate multiple sentiment classification models developed using machine learning techniques. The selected model is integrated into a real-time streaming pipeline constructed with Apache Kafka and Apache Spark, enabling continuous sentiment prediction on incoming review data. The processed results are stored in Elasticsearch and visualized through Kibana dashboards to provide insights into customer sentiment distribution and trends over time.

The scope of this project is limited to textual customer reviews obtained from the Google Play Store and focuses solely on sentiment analysis of the MAE application. Other data sources, such as social media platforms or customer support records are not included in this implementation.

2.0 Data Acquisition & Preprocessing

2.1 Data Source

The primary data source used in this project is customer reviews of the MAE by Maybank mobile banking application collected from the Google Play Store. The Google Play Store was selected because it contains authentic user-generated feedback that reflects customers' experiences with the application, including comments on usability, system performance, transaction reliability, security and overall user satisfaction.

Compared to social media platforms, Google Play reviews provide structured information such as review text, start rating, review date and application version. These attributes are highly suitable for sentiment analysis as they enable both textual analysis and comparison between user ratings and predicted sentiment classes.

The dataset was collected using the google-play-scraper Python library, which allows automated extraction of publicly available application reviews without requiring access to the Google Play Developer API. The reviews were filtered using the following configuration:

Parameter	Value
Application	MAE by Maybank
App ID	com.maybank2u.life
Country	Malaysia
Language	English
Sorting Method	Latest Reviews
Target Reviews	5,000

Although the initial target was to collect 5,000 reviews, the final dataset contained 4,155 unique reviews after removing duplicate records and collecting the latest available reviews that matched the specified parameters.

2.2 Development Tools and Libraries

Several software tools and Python libraries were used throughout the data acquisition and preprocessing stages. Each tool serves a specific purpose in the overall data engineering workflow.

Tool / Library	Purpose
Python 3.11	Main programming language
Visual Studio Code	Development environment
Google Play Scraper	Collect Google Play reviews
Pandas	Data manipulation and preprocessing

NLTK	Tokenization and stopword removal
spaCy	Lemmatization
Emoji	Emoji removal

Python was selected because of its extensive ecosystem of libraries for Natural Language Processing (NLP), machine learning and big data applications. Visual Studio Code served as the primary integrated development environment (IDE), while the Google Play Scraper library enabled efficient extraction of review data from the Google Play Store.

2.3 Data Collection Process

The data collection process consisted of several stages to ensure that the extracted reviews were complete, relevant and suitable for sentiment analysis.

Initially, the MAE by Maybank application was identified using its unique application ID (com.maybank2u.life). The Google Play Scraper library was then configured to retrieve the latest Malaysian reviews using the English language setting. Due to limitations on the number of reviews returned per request, the scraper employed pagination through continuation tokens to repeatedly retrieve additional batches of reviews until the desired number of records was obtained.

After collection, the retrieved data was converted into a Pandas DataFrame for further processing. Only the attributes relevant to the sentiment analysis task were retained, while unnecessary information such as user profile images and developer replies was excluded from the dataset.

The final dataset consists of the following attributes:

Attribute	Description
review_id	Unique identifier of each review
review	Original review text
rating	User rating (1 – 5 stars)
review_date	Date when the review was submitted
app_version	Application version

```
# Display the first 10 rows
df.sample(10)
```

✓ 0.0s

	review_id	review	rating	review_date	thumbs_up	app_version
19	8fe9eaf1-3660-4f0c-8d6b-f15c073cd5c5	Maybank2u is a reliable and easy-to-use bankin...	5	2026-06-26 22:40:04	0	NaN
20	b855ce68-9400-4796-876b-645995289e64	Apps ni melibatkan duit yang disimpan pelangga...	1	2026-06-26 21:58:17	0	NaN
87	f39c2af8-2a8e-4147-95f2-53c351e9e9cf	nice apps	5	2026-06-20 18:39:24	0	0.9.47
138	4a74dbd7-d78b-429f-acab-493720b9f0d6	transfer money is more easy here	5	2026-06-16 01:47:51	0	NaN
11	43e40ab0-4499-4dd2-964c-9b8e2b25c1cb	I've been using Maybank2u for daily banking li...	5	2026-06-27 22:04:24	0	NaN
68	bbc2ccfb-89b6-473d-a8e7-b28a140ada9f	ok good	5	2026-06-22 23:22:50	0	0.9.47
179	67d93d86-f5a2-446c-9b10-017bb1ae7556	good apps	5	2026-06-11 07:49:43	0	0.9.47
106	a935ea6f-ca62-4520-809e-832b623b1be7	MAE wallet and QR Pay make cashless life easy....	5	2026-06-18 21:33:23	0	0.9.47
141	09320027-aaa4-46a7-8439-e11f24c4bda4	why i can't change the transfer limit????	3	2026-06-15 13:24:08	0	0.9.47
113	084abe88-8ade-4a44-84a6-6339204beae7	Maybank2u is a highly practical mobile banking...	5	2026-06-18 14:17:04	0	NaN

Figure 2.3.1: Sample of the collected dataset.

2.4 Data Validation

Before preprocessing, several validation procedures were performed to evaluate the quality of the collected dataset.

Figure 2.4.1 shows that the dataset initially contained **4,155 customer reviews** distributed across five primary attributes. Data type verification confirmed that the review text and review identifier were stored as string values, while the rating was stored as an integer based on the Figure 2.4.2. The review date was initially imported as text and later converted into an appropriate format during preprocessing.

```
=====
DATASET OVERVIEW
=====
Number of rows      : 4155
Number of columns   : 5

Column Names:
['review_id', 'review', 'rating', 'review_date', 'app_version']
```

Figure 2.4.1: Dataset Overview


```

=====
DATASET INFORMATION
=====
<class 'pandas.DataFrame'>
RangeIndex: 4155 entries, 0 to 4154
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   review_id       4155 non-null   str
1   review          4155 non-null   str
2   rating          4155 non-null   int64
3   review_date     4155 non-null   str
4   app_version     3428 non-null   str
dtypes: int64(1), str(4)
memory usage: 863.8 KB

```

Figure 2.4.2: Dataset Information

Next, the Figure 2.4.3 shows that missing value analysis revealed that only the app_version attribute contained missing values, approximately **17.5%** of the dataset. Since this attribute is not required for sentiment classification, the corresponding records were retained to preserve as much textual information as possible.

```

=====
MISSING VALUE ANALYSIS
=====

```

	Missing Count	Missing Percentage (%)
review_id	0	0.0
review	0	0.0
rating	0	0.0
review_date	0	0.0
app_version	727	17.5

Figure 2.4.3: Missing Value Analysis

Otherwise, Figure 2.4.4 shows that the duplicate value analysis was performed using the review text as the reference attribute. No duplicate reviews were found after the cleaning process, ensuring that each review represented a unique customer opinion.

```

=====
DUPLICATE VALUE ANALYSIS
=====
Duplicate Reviews : 0

```

Figure 2.4.4: Duplicate Value Analysis

To obtain a better understanding of the collected data, the distribution of user ratings was also examined and the output shown in Figure 2.4.5. The analysis showed that one-star and five-star ratings dominated the dataset, indicating that users generally submitted reviews when they experienced either significant dissatisfaction or high satisfaction with the application.

```
=====
RATING DISTRIBUTION
=====
rating
1      1881
2       341
3       319
4       185
5      1429
Name: count, dtype: int64
```

Figure 2.4.5: Rating Distribution

2.5 Text Preprocessing

Raw customer reviews collected from the Google Play Store often contain inconsistent formatting, unnecessary symbols, emojis, hyperlinks, and other noisy textual information. Such inconsistencies may negatively affect the performance of Natural Language Processing (NLP) models by increasing vocabulary size and introducing irrelevant features. Therefore, several preprocessing techniques were applied to standardize the review text before sentiment classification.

2.5.1 Lowercase Conversion

The first preprocessing step converts all review text into lowercase characters. This ensures that words with different capitalization, such as "Good", "GOOD", and "good", are treated as the same token during text processing, reducing duplicate vocabulary.

```
# Convert all text to lowercase
df["clean_review"] = df["clean_review"].str.lower()
df[["review", "clean_review"]].head()
```

Figure 2.5.1.1: Code for Lowercase Conversion

	review	clean_review
0	A dependable banking app that makes handling m...	a dependable banking app that makes handling m...
1	very good app	very good app
2	Amazing service and great experience! 😊 Thank ...	amazing service and great experience! 😊 thank ...
3	I gave this app a chance it works up until the...	i gave this app a chance it works up until the...
4	Can't reload my phone credit through FPX, noti...	can't reload my phone credit through fpx, noti...

Figure 2.5.1.2: Output for Lowercase Conversion

2.5.2 URL Removal

Some customer reviews contain hyperlinks that do not contribute meaningful information for sentiment analysis. These URLs were removed using regular expressions to eliminate unnecessary noise from the dataset.

```
# Function to remove URLs from text
def remove_urls(text):
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)
    return text

# Apply the function to the 'clean_review' column
df["clean_review"] = df["clean_review"].apply(remove_urls)
```

Figure 2.5.2.1: Code for URL Removal

2.5.3 HTML Tag Removal

HTML tags occasionally appear in scraped text data. Since these tags do not represent customer opinions, they were removed using regular expression matching.

```
# Function to remove HTML tags from text
def remove_html(text):
    text = re.sub(r"<.*?>", "", text)
    return text

# Apply the function to the 'clean_review' column
df["clean_review"] = df["clean_review"].apply(remove_html)
```

Figure 2.5.3.1: Code for HTML Tag Removal

2.5.4 Emoji Removal

Many users express emotions through emojis. However, the sentiment classification models used in this project rely primarily on textual information. Therefore, emojis were removed to simplify the input text and reduce unnecessary symbols.

```
# Function to remove emojis from text
def remove_emojis(text):
    return emoji.replace_emoji(text, replace="")

# Apply the function to the 'clean_review' column
df["clean_review"] = df["clean_review"].apply(remove_emojis)
```

Figure 2.5.4.1: Code for Emoji Removal

	review	clean_review
984	masalah	masalah
2497	Lambat dan lag bila nak cepat tpi bila xde apa...	lambat dan lag bila nak cepat tpi bila xde apa...
534	exellent	exellent
181	now very good 🍑 😊	now very good

Figure 2.5.4.2: Output for Emoji Removal

2.5.5 Punctuation Removal

Punctuation marks such as commas, periods, question marks, and exclamation marks do not generally contribute to sentiment classification. These characters were removed to produce cleaner textual data.

```
# Function to remove punctuation from text
def remove_punctuation(text):
    return text.translate(str.maketrans("", "", string.punctuation))

# Apply the function to the 'clean_review' column
df["clean_review"] = df["clean_review"].apply(remove_punctuation)
```

Figure 2.5.5.1: Code for Punctuation Removal

	review	clean_review
3256	memudahkan pengguna	memudahkan pengguna
3513	Everytime update update.	everytime update update
3847	Developer sial.Dah 8 bulan aku bersabar dengan...	developer sialdah 8 bulan aku bersabar dengan ...
2210	before this can use but the latest update rest...	before this can use but the latest update rest...
3824	Assalamualaikum en, nk tnya. Awat otp saya xdi...	assalamualaikum en nk tnya awat otp saya xdite...

Figure 2.5.5.2: Output for Punctuation Removal

2.5.6 Extra Space Removal

After removing URLs, emojis, and punctuation, multiple spaces may remain within the text. Extra whitespace was removed to ensure that the cleaned reviews follow a consistent format.

```
# Function to remove extra spaces from text
def remove_extra_spaces(text):
    return " ".join(text.split())

# Apply the function to the 'clean_review' column
df["clean_review"] = df["clean_review"].apply(remove_extra_spaces)
```

Figure 2.5.6.1: Code for Extra Space Removal

2.5.7 Tokenization

Tokenization divides each review into individual words, commonly referred to as tokens. This process enables subsequent NLP operations such as stopwords removal and lemmatization.

```
# Function to tokenize text
def tokenize_text(text):
    return word_tokenize(text)

✓ 0.0s

# Apply the function to the 'clean_review' column
df["tokens"] = df["clean_review"].apply(tokenize_text)
```

Figure 2.5.7.1: Code for Tokenization

	clean_review	tokens
0	a dependable banking app that makes handling m...	[a, dependable, banking, app, that, makes, han...
1	very good app	[very, good, app]
2	amazing service and great experience thank you...	[amazing, service, and, great, experience, tha...
3	i gave this app a chance it works up until the...	[i, gave, this, app, a, chance, it, works, up,...
4	cant reload my phone credit through fpx notifi...	[cant, reload, my, phone, credit, through, fpx...

Figure 2.5.7.1: Output for Tokenization

2.5.8 Stopword Removal

Common English stopwords obtained from the NLTK library were combined with a custom list of Malay stopwords. These words generally contribute little semantic meaning and may reduce model performance if retained.

A custom Malay stopwords list was used because the collected reviews contain a mixture of English and Malay. However, sentiment-related words such as "tak", "tidak", and "boleh" were intentionally preserved because removing them could alter the sentiment expressed in a review.

```
english_stopwords = set(stopwords.words("english"))
✓ 0.0s

malay_stopwords = {
    "yang", "dan", "di", "ke",
    "dari", "untuk", "ini", "itu",
    "ni", "lah", "pun", "je",
    "jer", "dengan", "pada",
    "dalam", "oleh", "atau"
}
```

```

all_stopwords = english_stopwords.union(malay_stopwords)

# Function to remove stopwords from tokenized text
def remove_stopwords(tokens):
    return [
        word
        for word in tokens
        if word not in all_stopwords
    ]

# Apply the function to the 'tokens' column
df["tokens"] = df["tokens"].apply(remove_stopwords)

```

Figure 2.5.8.1: Code for Stopword Removal

	clean_review	tokens
0	a dependable banking app that makes handling m...	[dependable, banking, app, makes, handling, mo...
1	very good app	[good, app]
2	amazing service and great experience thank you...	[amazing, service, great, experience, thank, e...
3	i gave this app a chance it works up until the...	[gave, app, chance, works, restricted, apps, a...
4	cant reload my phone credit through fpx notifi...	[cant, reload, phone, credit, fpx, notificatio...

Figure 2.5.8.2: Output for Stopword Removal

2.5.9 Lemmatization

Lemmatization converts words into their base or dictionary form. This process reduces vocabulary size while preserving the original semantic meaning, allowing different grammatical forms of the same word to be treated as a single feature.

```

# Function to lemmatize tokens using spaCy
def lemmatize_tokens(tokens):
    text = " ".join(tokens)
    doc = nlp(text)

    return [token.lemma_ for token in doc]

# Apply the function to the 'tokens' column
df["tokens"] = df["tokens"].apply(lemmatize_tokens)

df["clean_review"] = df["tokens"].apply(lambda x: " ".join(x))

```

Figure 2.5.9.1: Code for Lemmatization

After all preprocessing steps were completed, the cleaned reviews were exported to `cleaned_data.csv`, which serves as the primary input dataset for the sentiment classification models developed in the next phase of the project. The final dataset retains both the original review and the cleaned review, enabling comparison between raw and processed text while supporting reproducibility and further analysis.

3.0 Sentiment Model Development

3.1 Dataset Preparation

The sentiment models were built using the clean dataset obtained from the data engineering process. The original dataset had 4,155 reviews about MAE mobile banking app obtained from the Google Play store. In the pre-processing stage, 25 reviews turned out to be empty since they only comprised of emojis, symbols or punctuation marks. These reviews were then eliminated due to lack of actual text, hence leaving behind a dataset of 4,130 reviews.

Sentiment labels were not available in the dataset, hence the labels were formed by transforming the ratings into three sentiments based on the following criteria:

Table 3.1: Star rating to sentiment label mapping

Star Rating	Sentiment Label
1-2 Stars	Negative
3 Stars	Neutral
4-5 Stars	Positive

Such kind of method is widely adopted in sentiment analysis in the absence of manually labeled data because the ratings of stars always denote the user's attitude towards the app.

The obtained distribution of sentiment is presented below:

Table 3.2: Sentiment class distribution

Sentiment	Count	Percentage
Negative	2,221	53.8%
Positive	1,590	38.5%
Neutral	319	7.7%

```
sentiment
negative    2221
positive    1590
neutral      319
Name: count, dtype: int64

Total usable reviews: 4130
```

Figure 3.1: Sentiment class distribution derived from MAE app star rating

From Figure 3.1, the data is imbalanced as there are more negatives compared to other classes. The negatives represent more than 50% of the data while the neutral class makes up 7.7% of the data. It is important for consideration since machine learning algorithms prefer majority classes during learning. Hence, class imbalance should be taken into account when building and evaluating the sentiment classification models.

3.2 Data Splitting

Prior to the model training, the dataset was split into two groups, the training and testing sets, following an 80-20 ratio, which give 3,304 and 826 reviews for training and testing purposes respectively.

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Train: {len(x_train)} | Test: {len(x_test)}")

{'negative': 0, 'neutral': 1, 'positive': 2}
Train: 3304 | Test: 826
```

Figure 3.2: Training and testing dataset split using stratified sampling

As shown in Figure 3.2, the stratified sampling method was used to ensure that the ratio between negative, neutral and positive reviews was similar in the two sets. This is significant since the neutral class is relatively small. In the absence of stratification, the class ratios might differ randomly and hence make the evaluation inconsistent. A fixed value of `random_state=42` was also specified to facilitate reproducibility of the experiment in the future.

3.3 Model 1: Naïve Bayes with TF-IDF

3.3.1 Feature Extraction

The Multinomial Naïve Bayes classification requires numerical input rather than the text itself. Hence, the text reviews have been transformed into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization.

The TF-IDF algorithm calculates the significance of each word within a review by assigning greater weights to those words which occur frequently in a specific review but are relatively less frequent in the whole dataset. This ensures that words that matter more in determining sentiment classes are given priority by the algorithm.

The vectorizer has been set up with 5,000 features (words) vocabulary size and also included unigrams as well as bigrams (`ngram_range=(1,2)`) as shown in Figure 3.3. The model is capable not only of recognizing individual words but also two-word combinations such as ‘tidak boleh’ or ‘very smooth’ because such combinations usually carry a stronger sentiment value than individual words.

```
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

Figure 3.3: TF-IDF vectorization for feature extraction

In other words, each review has been expressed in terms of a sparse numerical feature vector after vectorization.

3.3.2 Model Training

The Multinomial Naïve Bayes model was built using the TF-IDF features.

```
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
```

Figure 3.4: Training the multinomial Naïve Bayes model

Naïve Bayes is among the most popular techniques employed for text classification since it is easy and quick to implement, as well as effective in dealing with high dimensional textual data. When compared to the sophisticated deep learning models, Naïve Bayes is computationally inexpensive and provides comparable results in many cases of sentiment classification. That is why Naïve Bayes was chosen as the baseline model in this project. After the training process was complete, the model was tested using the unseen test dataset.

3.3.3 Evaluation

Naïve Bayes model was tested with the test dataset consisting of 82 reviews. Classification report is shown in Figure 3.5 below.

```
=== Naive Bayes Results ===
Accuracy: 0.7869
```

	precision	recall	f1-score	support
negative	0.73	0.96	0.83	444
neutral	0.00	0.00	0.00	64
positive	0.91	0.70	0.79	318
accuracy			0.79	826
macro avg	0.55	0.55	0.54	826
weighted avg	0.75	0.79	0.75	826

Figure 3.5: Classification report for the Naïve Bayes model

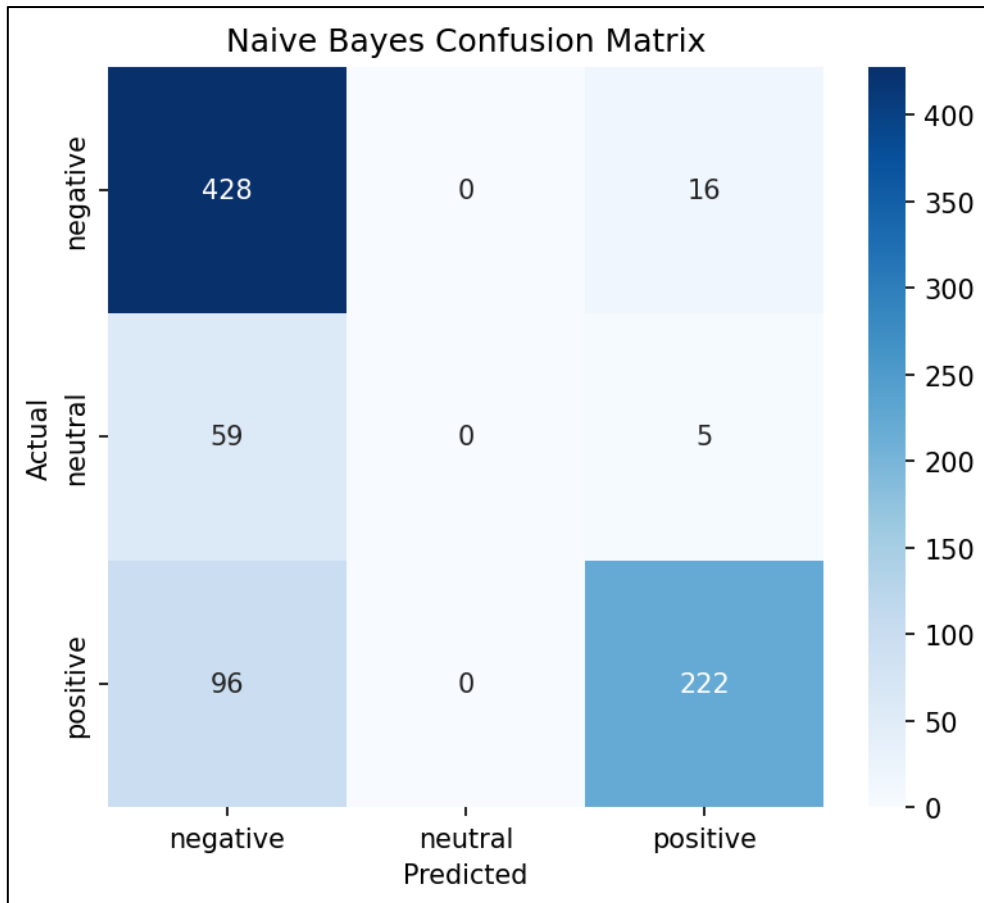


Figure 3.6: Confusion matrix for the Naïve Bayes model

Naïve Bayes model demonstrated 78.69% of accuracy rate that means that it managed to predict the classes correctly for the most part of the test data. Nevertheless, this indicator cannot be the only one to evaluate the model performance. Analyzing the performance on individual classes we can say that the model predicted the negative class perfectly and it showed quite good results while predicting the positive class with the precision of 0.91 and F1-score of 0.79. That means that it performed well on predicting the majority sentiment classes.

Unfortunately, the model completely failed to predict the neutral class since the precision, recall and F1-score of the neutral class equals to 0.00 which means that none of neutral reviews were correctly predicted. Confusion matrix shows that neutral reviews were often misclassified as negative or positive.

The behaviour is mostly determined by class imbalance that was observed during the dataset preparation process. Namely, since neutral class makes up 7.7% of the dataset the model learned patterns of the majority classes better and completely ignored the minority class.

Despite the fact that the performance of the Naïve Bayes classifier was higher in comparison to others with respect to accuracy, its failure to recognize neutral reviews indicates that accuracy is not a good indicator on its own and other metrics like precision, recall and F1 measure should be taken into account.

3.4 Model 2: Long Short-Term Memory (LSTM) Network

3.4.1 Text Tokenization and Sequence Encoding

As opposed to the Naïve Bayes algorithm, which relies on the TF-IDF features, the LSTM model handles texts as a series of words. Thus, the LSTM model can learn the context and dependencies of the words, hence being more appropriate to comprehend the structure of a sentence.

Prior to training the LSTM model, each review was transformed into a list of integers by means of the Keras Tokenizer method. Each unique word had its integer index and all reviews had their lengths set to 50 words.

```
MAX_VOCAB = 5000
MAX_LEN = 50

tokenizer = Tokenizer(num_words=MAX_VOCAB, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = pad_sequences(tokenizer.texts_to_sequences(X_train), maxlen=MAX_LEN,
                             padding="post", truncating="post")
X_test_seq = pad_sequences(tokenizer.texts_to_sequences(X_test), maxlen=MAX_LEN,
                             padding="post", truncating="post")
```

Figure 3.7: Text tokenization and sequence encoding using the Keras Tokenizer

Vocabulary size was set at 5,000 words as it would allow inclusion of common words but keep the model efficient. The length of the sequence was set at 50 words as majority of the reviews in the dataset were quite short with average length of about eight words.

3.4.2 Handling Class Imbalance

As highlighted in Section 3.1, there are significantly more neutral reviews than negative and positive reviews. In this case, there would be no way for the model to learn in favor of the minority classes without some sort of intervention.

Class weighting was performed on this dataset to solve this challenge. The reviews in the minority class were weighted highly such that the model could learn about them better.

```
y_train_cat = to_categorical(y_train, num_classes=3)
y_test_cat = to_categorical(y_test, num_classes=3)

# Compensate for the neutral class being small
weights = compute_class_weight("balanced", classes=np.unique(y_train), y=y_train)
class_weights = dict(enumerate(weights))
print(class_weights)

{0: 0.6197711498780717, 1: 4.318954248366013, 2: 0.8658280922431866}
```

Figure 3.8: Computed class weights for handling class imbalance

This is because the class with the largest weight is the one that contains neutral reviews, which indicates the least represented class as shown in Figure 3.8. With this penalty for incorrectly categorizing this particular type of review, the classifier can learn better decision boundaries that do not concentrate only on the dominant classes.

3.4.3 Model Architecture

The LSTM sentiment classification model was developed on top of a sequential neural network containing Embedding, LSTM, Dense and Dropout layers.

Layer (type)	Output shape	Param #
embedding (Embedding)	(None, 50, 128)	640,000
lstm (LSTM)	(None, 64)	49,408
dense (Dense)	(None, 32)	2,080
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 3)	99

Figure 3.9: Summary of the LSTM network architecture

Each word is mapped to a dense vector by the Embedding layer so that similar words will have similar representations. The vectors are passes through the LSTM layer in order to learn the relationships among the words for each review. It allows the model to capture the word order and sentence context much better than machine learning. In order to avoid overfitting, the Dropout technique was used inside the LSTM layer as well as after hidden Dense layer.

3.4.4 Model Training

For the training of the LSTM model, the Adam optimizer and the categorical cross entropy loss function were utilized. The number of epochs to train the network was set at a maximum of 15 and the batch size was set to be 32. In order to avoid over-fitting of the model, Early Stopping was applied where training will stop automatically when there is no improvement in validation loss for three consecutive epochs.

```

Epoch 1/15
93/93 ————— 4s 20ms/step - accuracy: 0.3414 - loss: 1.1003 - val_accuracy: 0.0906 - val_loss: 1.1065
Epoch 2/15
93/93 ————— 2s 16ms/step - accuracy: 0.3269 - loss: 1.0993 - val_accuracy: 0.5438 - val_loss: 1.0874
Epoch 3/15
93/93 ————— 2s 16ms/step - accuracy: 0.3690 - loss: 1.1009 - val_accuracy: 0.3867 - val_loss: 1.0884
Epoch 4/15
93/93 ————— 2s 19ms/step - accuracy: 0.5668 - loss: 1.0052 - val_accuracy: 0.7462 - val_loss: 0.9272
Epoch 5/15
93/93 ————— 3s 30ms/step - accuracy: 0.6815 - loss: 0.9025 - val_accuracy: 0.7462 - val_loss: 0.9153
Epoch 6/15
93/93 ————— 3s 31ms/step - accuracy: 0.6889 - loss: 0.8735 - val_accuracy: 0.7704 - val_loss: 0.8930
Epoch 7/15
93/93 ————— 3s 28ms/step - accuracy: 0.7249 - loss: 0.8671 - val_accuracy: 0.7795 - val_loss: 0.8843
Epoch 8/15
93/93 ————— 3s 32ms/step - accuracy: 0.7679 - loss: 0.8569 - val_accuracy: 0.7704 - val_loss: 0.8851
Epoch 9/15
93/93 ————— 3s 34ms/step - accuracy: 0.7656 - loss: 0.8376 - val_accuracy: 0.7795 - val_loss: 0.8695
Epoch 10/15
93/93 ————— 3s 35ms/step - accuracy: 0.7353 - loss: 0.8302 - val_accuracy: 0.7795 - val_loss: 0.8747
Epoch 11/15
93/93 ————— 3s 31ms/step - accuracy: 0.7807 - loss: 0.8281 - val_accuracy: 0.7825 - val_loss: 0.8833
Epoch 12/15
93/93 ————— 3s 29ms/step - accuracy: 0.7077 - loss: 0.8340 - val_accuracy: 0.7855 - val_loss: 0.8369
Epoch 13/15
...
Epoch 14/15
93/93 ————— 3s 30ms/step - accuracy: 0.7053 - loss: 0.8323 - val_accuracy: 0.7613 - val_loss: 0.8929
Epoch 15/15
93/93 ————— 3s 28ms/step - accuracy: 0.7171 - loss: 0.8240 - val_accuracy: 0.7644 - val_loss: 0.8915

```

Figure 3.10: LSTM training log showing epoch-wise accuracy and loss

Both the training and validation metrics were tracked during training for each epoch. As indicated in Figure 3.10, the model showed improvement in terms of reduction in training loss and increase in validation accuracy as the training progresses. The training process was carried out for the entire 15 epochs using the optimal weight values obtained from the 12th epoch. At this point, the lowest validation loss value of 0.8369 was obtained. The model performed stably during the whole process, with a steady improvement of the validation accuracy from 0.09 at the 1st epoch to 0.7855 at the 12th epoch.

3.4.5 Evaluation

The trained LSTM model was evaluated using the same test dataset consisting of 826 reviews.

```

=== LSTM Results ===
Accuracy: 0.7663

```

	precision	recall	f1-score	support
negative	0.72	0.95	0.82	444
neutral	0.00	0.00	0.00	64
positive	0.89	0.66	0.76	318
accuracy			0.77	826
macro avg	0.54	0.54	0.53	826
weighted avg	0.73	0.77	0.73	826

Figure 3.11: Classification report for the LSTM model

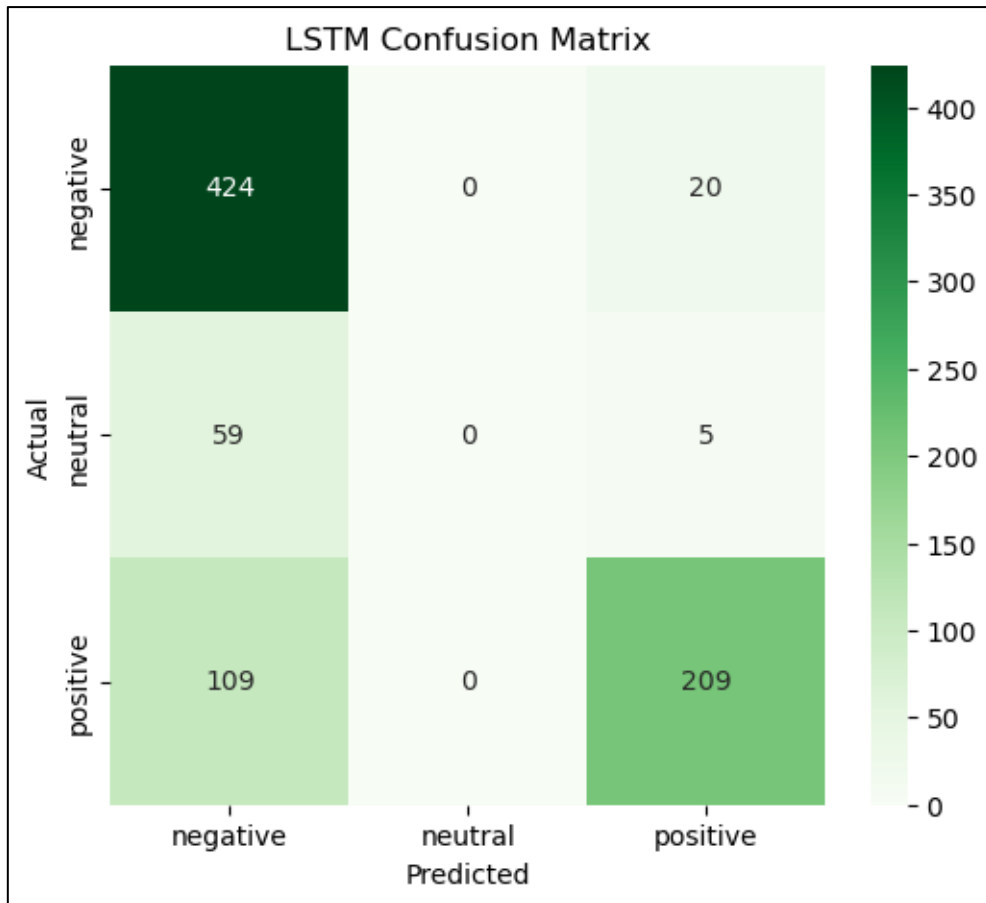


Figure 3.12: Confusion matrix for the LSTM model

LSTM model had an accuracy of 76.63%, which is relatively low when compared to the Naïve Bayes model. The model managed to classify negative reviews effectively, having a recall of 0.95, while precision of the positive class also had a good result of 0.89. Thus, it could be concluded that the model performed effectively in classifying the two majority sentiment classes.

Similar to the Naïve Bayes model, LSTM model also failed to recognize the neutral class with all its precision, recall and F1-score being equal to 0.00. This could mean that the model remained highly biased towards the imbalanced data despite the usage of the class weights. Mostly, neutral reviews got predicted either as negative or positive reviews.

Thus, it could be seen that despite the decent performance of LSTM model in terms of the majority classes. It did not manage to classify better the minority neutral class.

3.5 Model Comparison

Table 3.3: Model comparison table

Metric	Naïve Bayes	LSTM
Overall Accuracy	0.79	0.77
Macro F1-Score	0.54	0.53
Negative Recall	0.96	0.95
Neutral Recall	0.00	0.00
Positive Recall	0.70	0.66
Positive Precision	0.91	0.89
Training Time	Very fast (Less than 1 second)	~45s (15 epochs)

Based on this comparison, it is evident that both classifiers had comparable accuracy levels on the test dataset. The Naïve Bayes model exhibited slightly better performance by having an accuracy level of 79% compared to the 77% accuracy level of the LSTM classifier. Both models showed great efficiency in detecting negative reviews but were not able to categorize the neutral class, with a recall rate of 0.00. This means that class imbalance became a problem for both classifiers.

The LSTM model can learn contextual information through training on text data. However, it took a long time to train and did not do better than the Naïve Bayes model. On average, the Naïve Bayes model performs better in classifying texts, whereas the LSTM model has the added benefit of being able to capture sequential information between words. The choice of which model to use is presented in the next chapter.

4.0 Apache System Architecture

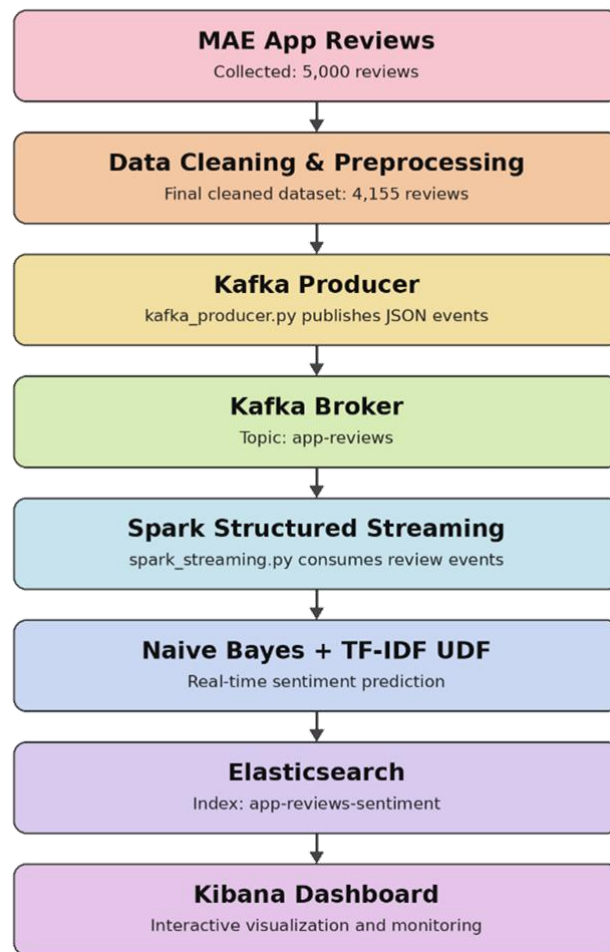


Figure 4.1 Kafka + Spark + Elasticsearch Workflow Diagram

Figure 4.1 illustrates the end-to-end real-time sentiment analysis architecture developed for this project. The process begins with 5,000 MAE mobile banking reviews collected from the Google Play Store. After data cleaning and preprocessing, 4,155 reviews remained in the final cleaned dataset. The Kafka producer reads the cleaned reviews and publishes them as JSON messages to the app-reviews Kafka topic. Spark Structured Streaming continuously consumes the incoming messages and applies a User Defined Function (UDF) that uses the trained TF-IDF vectorizer and Naive Bayes model to classify each review as positive, negative or neutral. Although both Naive Bayes and LSTM models were evaluated during model development, Naive Bayes was selected for deployment because it achieved slightly higher classification accuracy while providing faster and more efficient inference for streaming applications. The classified results are then stored in Elasticsearch under the app-reviews-sentiment index. Finally, Kibana is used to visualize the stored sentiment results through an interactive dashboard for real-time monitoring and analysis.

5.0 Analysis & Results

The data collection and preprocessing stages produced 4,155 cleaned MAE by Maybank app reviews from the original 5,000 collected reviews. During text preprocessing for model training, 25 reviews became empty after removing emojis, punctuation, symbols, and other non-textual content. These records were excluded from the model development process, resulting in 4,130 usable reviews for training and evaluation. The final training dataset was labelled into three sentiment classes based on star ratings: negative (2,221), positive (1,590), and neutral (319). This distribution is highly imbalanced, with neutral reviews accounting for less than 8% of the dataset, which significantly influenced the classification performance of both machine learning models.

Two models were trained on an 80/20 stratified split (3,304 train / 826 test):

Naive Bayes (TF-IDF, 5,000 features, unigrams + bigrams)

- Overall accuracy: **78.69%**
- Negative: precision 0.73, recall 0.96, F1 0.83
- Positive: precision 0.91, recall 0.70, F1 0.79
- Neutral: precision 0.00, recall 0.00, F1 0.00

LSTM (embedding dim 128, 64-unit LSTM, class-weighted training, early stopping)

- Overall accuracy: **76.63%**
- Negative: precision 0.72, recall 0.95, F1 0.82
- Positive: precision 0.89, recall 0.66, F1 0.76
- Neutral: precision 0.00, recall 0.00, F1 0.00

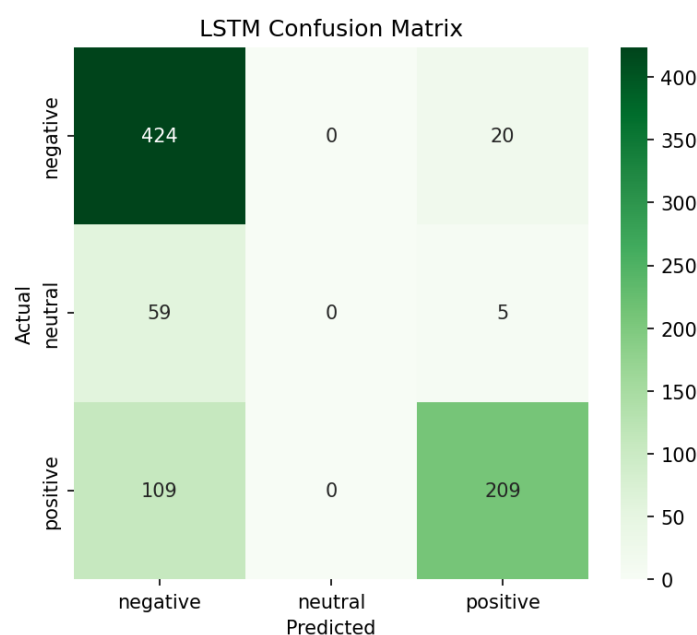


Figure 5.1 LSTM Confusion Matrix

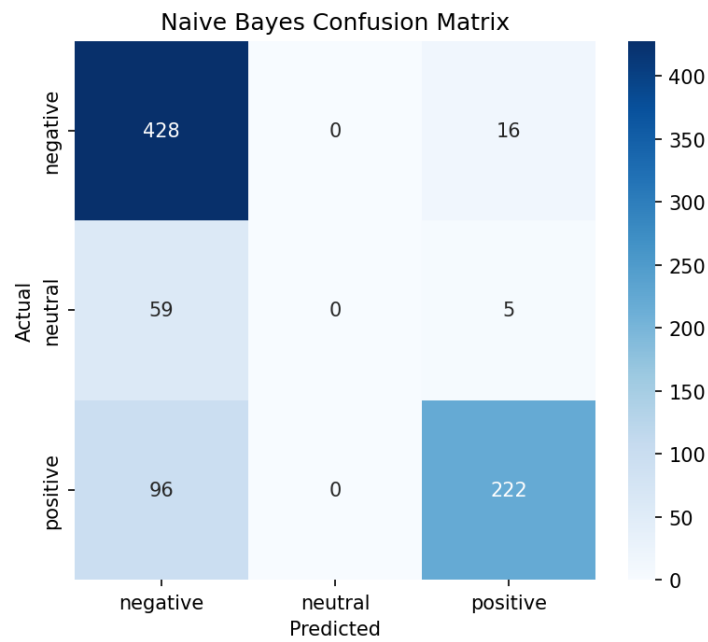


Figure 5.2 Naive Bayes Confusion Matrix

Both confusion matrices show a similar classification pattern. Negative reviews achieved high recall, while some positive reviews were incorrectly classified as negative. Neither model correctly identified the neutral class, as all neutral reviews were assigned to either the negative or positive category. This result is mainly caused by the severe class imbalance, because neutral reviews represent only a small proportion of the training dataset. The Naive Bayes model achieved a slightly higher overall accuracy of 78.69% compared with 76.63% for the LSTM model. It also required significantly less training time and computational resources. Therefore, the Naive Bayes model was selected for integration into the Kafka and Spark real-time streaming pipeline.

6.0 Optimization & Comparison

6.1 Naive Bayes vs LSTM

Table 6.1 Naive Bayes vs LSTM Comparison

Aspect	Naive Bayes	LSTM
Accuracy	78.69%	76.63%
Macro F1	0.54	0.53
Training time	< 1 Seconds	~10 epochs, 45 seconds (early-stopped)
Model size	Lightweight (TF-IDF + MultinomialNB)	2.07M parameters (7.91 MB)
Inference cost	Very low, sparse vector dot product	Higher, requires sequence padding + tensor forward pass
Streaming suitability	Loads once per worker, cheap per-row inference	Heavier per-row inference, less ideal for high-throughput streams

Despite the LSTM's greater representational capacity, Naive Bayes edged it out on both accuracy and macro F1 in this run. Two things explain this rather than deep learning failing generally:

1. **Dataset size.** 3,304 training examples is small for a neural sequence model to outperform a well-tuned bag-of-words baseline — LSTMs typically need substantially more data to leverage word-order information.
2. **Review length and vocabulary.** Short, mostly template-like app reviews reward simple lexical cues (e.g., "good," "problem," "cannot login") that TF-IDF captures directly, with less benefit from sequential context.

For this reason, the pipeline deployed to the Kafka/Spark streaming layer uses the **Naive Bayes + TF-IDF** model rather than the LSTM: it is both more accurate on this dataset and far cheaper to serialize and reload inside a Spark UDF, which matters when the model is loaded once per executor and applied per streamed message. The LSTM remains saved (`lstm_model.keras`, `lstm_tokenizer.pkl`) as a benchmark and for potential future use once more training data is available.

6.2 Compare model and pipeline performance between batch mode and streaming mode

Table 6.2 Batch vs. Streaming Performance Comparison

Metric	Batch Mode	Streaming Mode (per-record)
Test set size	826 reviews	826 reviews
Total processing time	15.03 ms	404.25 ms
Throughput	~54,941 records/sec	~2,043 records/sec
Avg. latency per record	0.018 ms	0.489 ms
P95 latency per record	—	0.618 ms
Accuracy	78.69%	78.69%

Key findings:

- **Accuracy is identical** between the two modes (78.69%) — this is expected, since it's the same trained, deterministic Naive Bayes model. Batch vs. streaming doesn't change *what* the model predicts, only *how fast* it predicts.
- **Batch mode is ~27x more efficient per record.** This is because TF-IDF vectorization and prediction are vectorized operations — transforming 826 reviews in one matrix operation is far cheaper than 826 separate function calls, each paying fixed overhead for tokenization, sparse matrix construction, and Python call overhead.
- **Streaming still comfortably meets real-time requirements.** Even at ~0.5 ms per record worst-case, the pipeline can classify roughly 2,000 reviews/sec — far beyond the arrival rate of real app review traffic, so the per-record overhead is a non-issue in practice.

this benchmark ran the model locally in Python to isolate *inference* efficiency. A live Spark Structured Streaming deployment would sit somewhere between these two numbers — it doesn't process one row per UDF call, it groups incoming Kafka messages into **micro-batches**, so it recovers most of the batch-mode vectorization efficiency while still delivering sub-second, near-real-time results.

7.0 Dashboard Visualization and Analysis

The classified sentiment results stored in Elasticsearch were visualized using Kibana to provide an interactive overview of customer feedback for the MAE by Maybank mobile application. The dashboard presents key sentiment metrics, rating distributions, temporal sentiment trends and keyword analysis through multiple visualizations. These dashboards enable stakeholders to monitor customer opinions in real time and identify recurring issues affecting user experience.

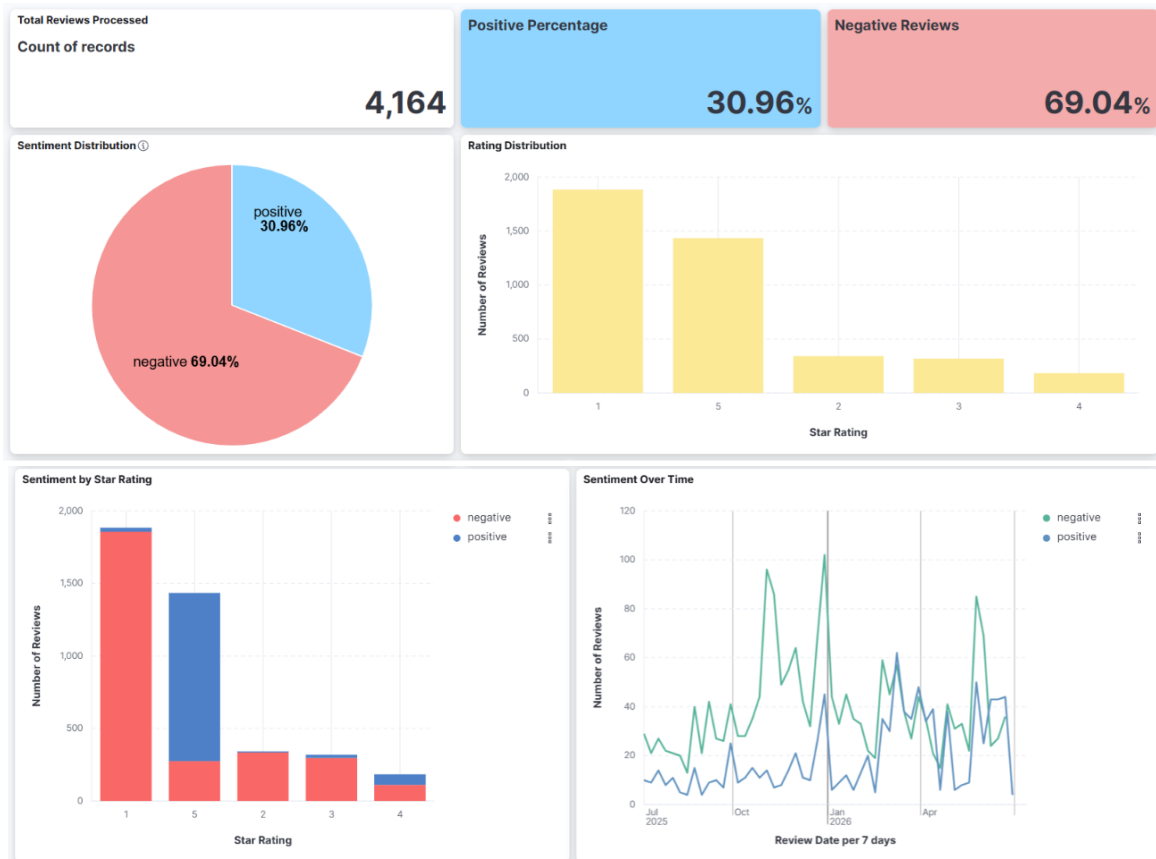


Figure 7.1: Kibana Dashboard for Real-Time Sentiment Analysis of MAE Mobile Application Reviews

The Total Reviews Processed metric displays the total number of review records successfully processed by the real-time streaming pipeline and stored in Elasticsearch. A total of 4,164 reviews were processed during the streaming simulation, demonstrating that the Kafka–Spark–Elasticsearch pipeline successfully ingested, classified, and stored review data for subsequent analysis.

The Positive Percentage and Negative Percentage metrics summarize the overall sentiment classification results. Approximately 30.96% of the processed reviews were classified as positive, whereas 69.04% were classified as negative. The accompanying pie chart further illustrates this distribution, showing that negative reviews dominate the dataset. This indicates that most users expressed dissatisfaction with the application, suggesting that improvements are required to enhance user experience and service quality.

The Sentiment by Star Rating visualization compares the predicted sentiment across different star ratings. Most one-star reviews are classified as negative, whereas five-star reviews are predominantly classified as positive. This consistency between user ratings and predicted sentiment demonstrates that the deployed Naive Bayes model produces sentiment classifications that align well with customer ratings.

A word cloud titled "Positive Reviews Word Cloud" featuring various terms in English and Indonesian. The most prominent words are "easy", "bank", "great", "good", "convenient", "fast", "reliable", "secure", "transfer", "interface", "pay", "well", "transaction", "check", "service", "money", "account", "platform", "friendly", "daily", "manage", "online", "feature", "allow", "recommend", "love", "simple", "nice", "useful", "efficient", "quick", "super", "anywhere", "really", "smooth", "update", "mobile", "fund", "highly", "user-friendly", "payment", "balance", "everyday", "helpful", "experience", "track", "malaysia", "mudah", "sangat", "ok", "overall", "anytime", "urusan", "security", "job", "cepat", "feel", "much", "finance", "platform", "friendly", "daily", "manage", "online", "feature", "allow", "recommend", "love", "simple", "nice", "useful", "efficient", "quick", "super", "anywhere", "really", "smooth", "update", "mobile", "fund", "highly", "user-friendly", "payment", "balance", "everyday", "helpful", "experience", "track", "malaysia", "mudah", "sangat", "ok", "overall", "anytime", "urusan", "security", "job", "cepat", "feel", "much", "finance", "platform", "friendly", "daily".

[illegible]

Figure 7.3: Negative Reviews Word Cloud

The Negative Reviews Word Cloud summarizes the most common terms appearing in negative customer reviews. Frequently occurring words include update, problem, password, phone, transaction, login, slow and fix, indicating that login failures, application updates, transaction issues and performance problems are among the major concerns reported by users. These insights can assist developers in identifying priority areas for future improvements.

Overall, the Kibana dashboard provides a comprehensive real-time overview of customer sentiment towards the MAE by Maybank mobile application. By integrating sentiment metrics, rating distributions, temporal trends and keyword visualizations, the dashboard enables stakeholders to quickly assess customer satisfaction, monitor sentiment changes and identify recurring issues requiring immediate attention. Combined with the Kafka, Spark Structured Streaming and Elasticsearch pipeline, the dashboard demonstrates an effective real-time sentiment monitoring solution capable of supporting data-driven decision-making.

8.0 Conclusion & Future Work

This project built an end-to-end sentiment analysis pipeline for MAE by Maybank app reviews: reviews were scraped via google-play-scraper, cleaned and labeled by star rating, used to train and compare a classical ML model (Naive Bayes) against a deep learning model (LSTM), and finally deployed into a real-time Kafka + Spark Structured Streaming pipeline that classifies incoming reviews on the fly. The Naive Bayes model was selected for production use based on its higher accuracy (78.69% vs 76.63%) and lower inference overhead, both important for a streaming context.

The clearest limitation is the **neutral class**: with only 319 examples (7.7% of the dataset), neither model learned to distinguish neutral sentiment from negative or positive, even with class weighting applied to the LSTM. This is a data problem, not a modeling problem, and it caps the ceiling on macro-averaged performance regardless of which algorithm is used.

Future work could include:

- **Rebalancing the dataset** — collecting more neutral-rated reviews specifically, or using techniques like SMOTE/oversampling or synthetic paraphrasing to boost the minority class.
- **Trying transformer-based models** (e.g., a fine-tuned DistilBERT or BERT-based sentiment classifier), which may better capture nuanced or sarcastic reviews than TF-IDF or a shallow LSTM.
- **Expanding the streaming pipeline** — adding a persistent sink (e.g., writing classified results to a database or dashboard instead of only console output), and scaling Kafka partitions for higher throughput.
- **Monitoring model drift** — since app reviews evolve as the app itself changes (new features, bugs, UI updates), periodically retraining on fresh data would keep the classifier relevant.
- **Multi-language support** — since MAE serves users in Malaysia, extending the pipeline to handle Bahasa Malaysia or code-mixed reviews would improve real-world coverage.

9.0 Reference

Presentation Video: <https://youtu.be/8MJF7TxYvgI>